

section .data

msg db "Sum of array elements: ", 10

msg_len equ \$-msg

[arr dq 10h, 20h,](#) 30h, 40h, 50h

n equ 5

%macro op 4

mov rax, %1

mov rdi, %2

mov rsi, %3

mov rdx, %4

syscall

%endmacro

section .bss

result resb 16

char_ans resb 16

section .text

global _start

_start:

op 1,1, msg, msg_len

mov rsi, arr

mov rcx, n

xor rax, rax

add_loop:

add rax, [rsi]

add rsi, 8

loop add_loop

; Convert sum in RAX to hexadecimal ASCII

mov rbx, rax

mov rdi, char_ans

mov rcx, 16

conv_loop:

rol rbx, 4

mov al, bl

```
and al, 0Fh
cmp al, 09h
jbe skip_add
add al, 07h
skip_add:
add al, 30h
mov [rdi], al
inc rdi
loop conv_loop
```

```
; Display the 16-digit result
op 1,1, char_ans, 16
```

```
; Exit
op 60, 0, 0, 0
```

section .data

msg1 db "Array sorted successfully", 10

len1 equ \$-msg1

arr dq 50h, 20h, 40h, 10h, 30h

n equ 5

%macro op 4

mov rax, %1

mov rdi, %2

mov rsi, %3

mov rdx, %4

syscall

%endmacro

section .bss

char_ans resb 16

newline db 10

```
section .text
global _start
_start:
mov rbx, n
dec rbx ; Outer loop counter = n - 1

outer_loop:
mov rcx, rbx ; Inner loop counter
mov rsi, arr

inner_loop:
mov rax, [rsi]
mov rdx, [rsi+8]
cmp rax, rdx
jle no_swap
; Swap elements
mov [rsi], rdx
mov [rsi+8], rax
```

no_swap:

add rsi, 8

loop inner_loop

dec rbx

jnz outer_loop

op 1,1, msg1, len1

; Exit

op 60, 0, 0, 0

section .data

prompt db "Original String: Hello World", 10

prompt_len equ

—promptmsg_end db 10, " *Length(Hex):* " *msg_len* equ
-msg_len

msg_rev db 10, "Reversed String: "

msg_rev_l equ \$-msg_rev

str db "Hello World", 0

%macro op 4

mov rax, %1

mov rdi, %2

mov rsi, %3

mov rdx, %4

syscall

%endmacro

section .bss

rev_str resb 32

len_buf resb 2

section .text

global _start

_start:

op [1,1](#), prompt, prompt_len

; 1. Calculate Length

mov rsi, str

xor rcx, rcx

len_calc:

cmp byte [rsi], 0

je len_done

inc rcx

inc rsi

jmp len_calc

len_done:

push rcx ; Save length

; Display Length label

op [1,1](#), msg_len, msg_len_l

; Display 2-digit hex length

mov rax, rcx

mov rbx, rax

and al, 0F0h

shr al, 4

cmp al, 9

jbe l1

add al, 7

l1: add al, 30h

mov [len_buf], al

mov al, bl

and al, 0Fh

cmp al, 9

jbe l2

add al, 7

```
l2: add al, 30h  
mov [len_buf+1], al
```

```
op 1,1, len_buf, 2
```

; 2. Reverse String

```
pop rcx ; Restore length  
mov rsi, str  
add rsi, rcx  
dec rsi ; Point to last char  
mov rdi, rev_str
```

```
rev_loop:  
mov al, [rsi]  
mov [rdi], al  
dec rsi  
inc rdi  
loop rev_loop
```

```
op 1,1, msg_rev, msg_rev_l
```

op [1,1](#), rev_str, 11

; Exit

op 60, 0, 0, 0

section .data

msg_gdtr db 10, "GDTR: "

l_gdtr equ

$-msg_gdtr$ msg_i dtr db 10, "IDTR: " l_i dtr equ

-msg_idtr

msg_ldtr db 10, "LDTR: "

l_ldtr equ $-msg_l$ dtr msg_t r db 10, "TR: " l_t $requ$

-msg_tr

msg_msw db 10, "MSW: "

l_msw equ \$-msg_msw

newline db 10

%macro op 4

mov rax, %1

mov rdi, %2

mov rsi, %3

mov rdx, %4

syscall

```
%endmacro
```

```
section .bss
```

```
gdt_buf resb 10
```

```
idt_buf resb 10
```

```
ldt_buf resw 1
```

```
tr_buf resw 1
```

```
msh_buf resd 1
```

```
char_ans resb 4
```

```
section .text
```

```
global _start
```

```
_start:
```

```
; Read system registers into buffers
```

```
sgdt [gdt_buf]
```

```
sidt [idt_buf]
```

```
sldt [ldt_buf]
```

```
str [tr_buf]
```

```
smsh [msh_buf]
```

; Display GDTR Limit (First 2 bytes)

op [1,1](#), msg_gdtr, l_gdtr

mov ax, [gdt_buf]

call disp_16

; Display IDTR Limit (First 2 bytes)

op [1,1](#), msg_idtr, l_idtr

mov ax, [idt_buf]

call disp_16

; Display LDTR

op [1,1](#), msg_ldtr, l_ldtr

mov ax, [ldt_buf]

call disp_16

; Display TR

op [1,1](#), msg_tr, l_tr

mov ax, [tr_buf]

call disp_16

; Display MSW (Lower 16 bits)

op [1,1](#), msg_msw, l_msw

mov ax, [msw_buf]

call disp_16

op [1,1](#), newline, 1

; Exit

op 60, 0, 0, 0

; Procedure to print 4-digit hex in AX

disp_16:

mov bx, ax

mov rdi, char_ans

mov rcx, 4

d_loop:

rol bx, 4

mov al, bl

and al, 0Fh

cmp al, 09h

```
jbe d_skip  
add al, 07h  
d_skip:  
add al, 30h  
mov [rdi], al  
inc rdi  
loop d_loop
```

```
op 1,1, char_ans, 4  
ret
```

```
section .data
```

```
msg_err db "Error opening file", 10
```

```
len_err equ $-msg_err
```

```
%macro op 4
```

```
mov rax, %1
```

```
mov rdi, %2
```

```
mov rsi, %3
```

```
mov rdx, %4
```

```
syscall
```

```
%endmacro
```

```
section .bss
```

```
fname resb 512
```

```
fd resq 1
```

```
buff resb 512
```

```
buflen resq 1
```

section .text

global _start

_start:

pop r8 ; argc

cmp r8, 2 ; Expecting 2 args (program name +
target file)

jne err

pop r8 ; argv[0]

pop r8 ; argv[1] (pointer to filename)

; Copy filename from stack into fname

mov rsi, fname

copy_fn:

mov al, [r8]

cmp al, 0

je do_open

mov [rsi], al

inc r8

inc rsi

jmp copy_fn

do_open:

; sys_open (syscall 2, O_RDONLY = 0)

op 2, fname, 0, 0777q

cmp rax, 0

jl err

mov [fd], rax

read_loop:

; sys_read (syscall 0)

op 0, [fd], buff, 512

cmp rax, 0

jle close_file

mov [bufflen], rax

; sys_write to stdout (syscall 1, fd 1)

op [1,1](#), buff, [bufflen]

jmp read_loop

close_file:

; sys_close (syscall 3)

op 3, [fd], 0, 0

jmp exit

err:

op [1,1](#), msg_err, len_err

exit:

op 60, 0, 0, 0

```
section .data
```

```
msg1 db "Error", 10
```

```
msg1l equ
```

```
–msg1msg2db" Success", 10msg2lequ -msg2
```

```
%macro op 4
```

```
mov rax, %1
```

```
mov rdi, %2
```

```
mov rsi, %3
```

```
mov rdx, %4
```

```
syscall
```

```
%endmacro
```

```
section .bss
```

```
fname1 resb 512
```

```
fd1 resq 1
```

```
fname2 resb 512
```

```
fd2 resq 1
```

buff resb 512

bufflen resq 1

section .text

global _start

_start:

pop r8

cmp r8, 3

jne err

pop r8

pop r8

mov rsi, fname1

above:

mov al, [r8]

cmp al, 00

je next

mov [rsi], al

inc r8

inc rsi

jmp above

next:

pop r8

mov rsi, fname2

above2:

mov al, [r8]

cmp al, 00

je next2

mov [rsi], al

inc r8

inc rsi

jmp above2

next2:

op 2, fname1, 0, 0777q

cmp rax, 0

jl err

mov [fd1], rax

op 0, [fd1], buff, 512

mov [bufflen], rax

; sys_creat (syscall 85)

op 85, fname2, 0777q, 0

; sys_open (syscall 2, O_WRONLY/RDWR = 2)

op 2, fname2, 2, 0777q

mov [fd2], rax

; sys_write to destination file

op 1, [fd2], buff, [bufflen]

op 3, [fd2], 0, 0

op 3, [fd1], 0, 0

op 1,1, msg2, msg2l

jmp end

err:

op 1,1, msg1, msg1l

end:

op 60, 0, 0, 0

